

Today 5:51 PM

i wanna do a recap on database sql and sql server tell me the points to recap

Absolutely. Since you already studied **DB / SQL / T-SQL** and you're preparing for **.NET backend interviews**, I'd structure the recap from fundamentals → SQL → SQL Server → advanced interview topics.

1. Database Fundamentals

- What is a database?
- DBMS vs RDBMS
- Tables, rows, columns
- Primary Key
- Foreign Key
- Candidate Key / Alternate Key
- Composite Key
- Surrogate Key
- Constraints
 - NOT NULL
 - UNIQUE
 - PRIMARY KEY
 - FOREIGN KEY
 - CHECK
 - DEFAULT
- Relationships
 - One-to-One
 - One-to-Many
 - Many-to-Many
- Normalization
 - 1NF
 - 2NF
 - 3NF
 - Denormalization

2. SQL Fundamentals

SELECT

- SELECT
- FROM
- WHERE
- DISTINCT
- TOP
- ORDER BY

Filtering

- Comparison operators
- AND , OR , NOT
- IN
- BETWEEN
- LIKE
- IS NULL
- IS NOT NULL

Aliases

- Column aliases
 - Table aliases
-

3. SQL Functions

Aggregate Functions

- COUNT
- SUM
- AVG
- MIN
- MAX

String Functions

- LEN
- LEFT
- RIGHT
- SUBSTRING
- CHARINDEX
- CONCAT
- LOWER
- UPPER

- TRIM

Date Functions

- GETDATE
- DATEADD
- DATEDIFF
- DATEPART
- YEAR
- MONTH
- DAY

NULL Handling

- ISNULL
 - COALESCE
 - Difference between NULL and empty string
-

4. GROUP BY & HAVING

Very important for interviews:

- GROUP BY
- HAVING
- Difference between WHERE and HAVING
- Aggregation with GROUP BY
- Multiple grouping columns

Example questions:

Find departments having more than 5 employees.

Find the average salary for each department.

5. JOINS ★★ ★

This is one of the most important sections.

Understand:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN

- FULL OUTER JOIN
- CROSS JOIN
- Self Join

And especially:

INNER vs LEFT JOIN

Understand what happens when there is **no matching record**.

Also practice joining 3–5 tables.

For example:

</> Plain text

```
Customer
  ↓
Order
  ↓
OrderItem
  ↓
Product
```

6. Subqueries

- Scalar subquery
- Subquery with IN
- Subquery with EXISTS
- Correlated subquery
- EXISTS vs IN

Important interview question:

When would you use EXISTS instead of IN ?

7. Set Operators

- UNION
- UNION ALL
- INTERSECT
- EXCEPT

Know the difference between:

</> Plain text

UNION

and

</> Plain text

UNION ALL

especially performance and duplicates.

8. SQL Commands / CRUD

DML

- INSERT
- UPDATE
- DELETE

DDL

- CREATE
- ALTER
- DROP
- TRUNCATE

DCL

- GRANT
- REVOKE
- DENY

TCL

- COMMIT
- ROLLBACK
- SAVEPOINT

Important:

DELETE vs TRUNCATE vs DROP

9. Views ★★

Understand:

- What is a View?
 - Why use Views?
 - Creating a View
 - Updating data through a View
 - View vs Table
 - View vs Stored Procedure
-

10. Stored Procedures ★★★

Very important for SQL Server interviews.

Recap:

- Creating Stored Procedures
- Parameters
- Input parameters
- Output parameters
- Return values
- Calling procedures
- IF
- CASE
- Variables
- Error handling
- Procedure vs Function

Also understand why stored procedures can be useful in some architectures.

11. User-Defined Functions

- Scalar-valued functions
 - Table-valued functions
 - Inline TVF
 - Multi-statement TVF
 - Function vs Stored Procedure
-

12. Transactions ★★★

Extremely important for backend development.

Understand:

</> Plain text

```
BEGIN TRANSACTION
COMMIT
ROLLBACK
```

And **ACID**:

- Atomicity
- Consistency
- Isolation
- Durability

Also:

- Transaction lifecycle
- Nested transactions
- Error handling with transactions

13. Isolation Levels ★★ ★

Know:

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable
- Snapshot

And the problems they solve:

- Dirty Read
- Non-Repeatable Read
- Phantom Read

This is a very common senior/intermediate SQL interview area.

14. Indexes ★★ ★

You specifically should spend time here.

Understand:

- What is an index?
- Why indexes improve reads
- Why indexes can hurt writes
- Clustered index
- Non-clustered index
- Composite index
- Unique index
- Covering index
- Included columns
- Index fragmentation
- Index maintenance

Most important:

Clustered vs Non-Clustered

Understand physically/logically what happens rather than just memorizing definitions.

15. Execution Plans ★ ★

Learn the basics of SQL Server execution plans:

- Table Scan
- Index Scan
- Index Seek
- Key Lookup
- Sort
- Hash Match
- Nested Loops
- Merge Join

And understand:

Why is an Index Seek usually preferable to a Table Scan?

16. CTEs ★ ★

Common Table Expressions:

</> Plain text

WITH ...

Understand:

- Basic CTE
 - CTE with aggregation
 - CTE with joins
 - Recursive CTE
 - CTE vs Subquery
 - CTE vs Temporary Table
-

17. Window Functions

Very important for modern SQL interviews.

Learn:

- OVER()
- PARTITION BY
- ORDER BY

Functions:

- ROW_NUMBER()
- RANK()
- DENSE_RANK()
- NTILE()
- LAG()
- LEAD()

Example interview problem:

Find the second-highest salary in every department.

18. Temporary Tables

Understand:

- Local temporary table #Table
- Global temporary table ##Table
- Table variables
- Temporary table vs table variable
- When to use each

19. Identity & Sequences

SQL Server specific:

- IDENTITY
- IDENTITY_INSERT
- SCOPE_IDENTITY()
- @@IDENTITY
- IDENT_CURRENT
- SEQUENCE

Especially know why:

</> Plain text

```
SCOPE_IDENTITY()
```

is generally safer than:

</> Plain text

```
@@IDENTITY
```

for retrieving a newly generated identity value.

20. Triggers ★ ★

Since you previously wanted this for interview practice, definitely recap it.

- What is a Trigger?
- DML triggers
- AFTER triggers
- INSTEAD OF triggers
- INSERTED table
- DELETED table
- Trigger vs Stored Procedure
- Advantages/disadvantages
- Why excessive triggers can become problematic

21. Cursors

Know:

- What is a cursor?
- Cursor lifecycle
- DECLARE
- OPEN
- FETCH
- CLOSE
- DEALLOCATE

Most importantly:

Why should you generally prefer set-based operations over cursors?

22. SQL Server Architecture Basics

For a SQL Server-specific recap:

- SQL Server Database Engine
- Database
- Schema
- Tables
- Pages
- Extents
- Data files .mdf
- Secondary data files .ndf
- Log files .ldf
- Transaction log
- Buffer pool

You don't need DBA-level depth, but understand the concepts.

23. SQL Server Security

Know the difference between:

⟨/⟩ Plain text

Login

User

Role

Permission

And:

- Authentication
 - Windows Authentication
 - SQL Server Authentication
 - Authorization
 - Server roles
 - Database roles
 - GRANT
 - DENY
 - REVOKE
-

24. Performance Optimization

Very important for a .NET backend developer.

Recap:

- Proper indexes
- Avoid `SELECT *`
- Filtering early
- Avoid unnecessary joins
- SARGable queries
- Execution plans
- Index Seek vs Scan
- Avoid unnecessary subqueries
- Pagination
- Query optimization
- Statistics
- Parameterized queries

Also understand the classic problem:

```
</> SQL
```

```
WHERE YEAR(OrderDate) = 2026
```

vs a more index-friendly date range.

25. Database Design

Practice designing databases from requirements.

For example:

E-commerce

Plain text

Customer
Product
Category
Order
OrderItem
Payment

Think about:

- PKs
- FKs
- Cardinality
- Constraints
- Normalization
- Indexes
- Cascade behavior

This is particularly useful for your **.NET/EF Core** work.

26. SQL + EF Core Connection

Because you're targeting **.NET backend**, also recap how your SQL knowledge maps to EF Core:

- DbSet
- LINQ → SQL translation
- IQueryable vs IEnumerable
- Tracking vs AsNoTracking
- Include
- Projection with Select
- JOIN
- GroupBy
- Pagination
- Transactions
- Concurrency
- Migrations

- Relationships
- Cascade delete
- Index configuration

And importantly:

Understand the SQL generated by EF Core rather than treating EF Core as a replacement for SQL knowledge.

Priority for Your Interview

If you're short on time, I'd prioritize like this:

Tier 1 — MUST KNOW

1. SELECT / WHERE / ORDER BY
2. JOINS
3. GROUP BY / HAVING
4. Aggregate functions
5. PK / FK / constraints
6. Normalization
7. Subqueries
8. CRUD
9. DELETE vs TRUNCATE vs DROP
10. Indexes
11. Transactions + ACID
12. Stored Procedures
13. Views

Tier 2 — VERY IMPORTANT

14. CTE
15. Window Functions
16. Isolation Levels
17. Execution Plans
18. Triggers
19. Functions
20. Temporary Tables
21. Query optimization
22. SQL Server architecture basics

Tier 3 — ADVANCED

23. Cursors
 24. Recursive CTE
 25. Locking/blocking/deadlocks
 26. Advanced indexing
 27. Statistics
 28. Partitioning
 29. Query Store
 30. Backup/restore concepts
-

How I suggest we do the recap

Rather than reading all of this as theory, we can go **group by group**, exactly like your previous C# and Angular recap:

Group 1: Database fundamentals

Group 2: SELECT + filtering

Group 3: Functions + aggregation

Group 4: JOINS

Group 5: Subqueries + CTE

Group 6: Window functions

Group 7: CRUD + DDL/DML/TCL

Group 8: Keys + constraints + normalization

Group 9: Views + Stored Procedures + Functions

Group 10: Transactions + ACID + Isolation

Group 11: Indexes + Execution Plans

Group 12: Triggers + Cursors

Group 13: SQL Server-specific topics

Group 14: Performance + optimization

Group 15: SQL interview problems

For each group, I can give you the important interview questions first, then concise answers, then practical SQL exercises.

ok let's go

Group 1 — Database Fundamentals

Let's start with the foundation. Focus on understanding the concepts, not memorizing definitions.

1. What is a Database?

A **database** is an organized collection of data that allows us to store, retrieve, update, and manage information efficiently.

2. What is DBMS?

DBMS (Database Management System) is software used to create and manage databases.

Examples:

- SQL Server
- MySQL
- PostgreSQL
- Oracle

3. DBMS vs RDBMS

DBMS

- General database management system.
- Doesn't necessarily use relationships between tables.

RDBMS

- Stores data in **tables**.
- Tables can have relationships.
- Usually follows relational principles.

SQL Server is an **RDBMS**.

4. What is a Table?

A table stores data in:

</> Plain text

Columns → define the properties

Rows → represent individual records

Example:

	Id	Name	Salary
	1	Ahmed	10000
	2	Khaled	12000

Id , Name , and Salary → columns

Each employee → row/record

5. Primary Key ★

A **Primary Key** uniquely identifies every row in a table.

Properties:

- Must be unique.
- Cannot be NULL .
- A table has one primary key constraint.
- It can consist of multiple columns → **composite primary key**.

Example:

</> Plain text

```
Employee
-----
Id ← Primary Key
Name
Salary
```

6. Foreign Key ★

A **Foreign Key** creates a relationship between two tables.

Example:

</> Plain text

```
Department
-----
Id (PK)
Name

Employee
-----
Id (PK)
Name
DepartmentId (FK)
```

Employee.DepartmentId references Department.Id .

This ensures **referential integrity**.

7. Primary Key vs Foreign Key

Primary Key

Identifies a row

Unique

Cannot be NULL

Defines entity identity

Foreign Key

References another table

Can contain duplicates

Can be NULL depending on relationship/design

Defines relationship

8. What is a Composite Key?

A key consisting of **more than one column**.

Common example:

</> Plain text

OrderItem

OrderId

ProductId

Quantity

OrderId + ProductId could together form the primary key.

Individually they aren't unique, but together they are.

9. Constraints ★

Constraints enforce rules on your data.

Important SQL Server constraints:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE

- NOT NULL
- CHECK
- DEFAULT

Example:

</> Plain text

Age CHECK (Age >= 18)

means invalid ages can't be inserted.

10. UNIQUE vs PRIMARY KEY

Both enforce uniqueness, but:

Primary Key

- Identifies the row.
- Only one primary key constraint per table.
- Cannot contain NULL.

UNIQUE

- Enforces uniqueness on a column/set of columns.
- A table can have multiple UNIQUE constraints.
- NULL handling differs in SQL Server and deserves attention.

11. Relationships ★★ ★

One-to-One

</> Plain text

Person 1 — 1 Passport

One person has one passport.

One-to-Many

</> Plain text

Department 1 — * Employees

One department has many employees.

This is extremely common.

Many-to-Many

</> Plain text

Student * — * Course

You normally solve this using a **junction table**:

</> Plain text

```
Student
  ↓
StudentCourse
  ↑
Course
```

12. Normalization ★★ ★

Normalization organizes data to reduce:

- duplication
- inconsistency
- update problems

The important levels:

1NF

- Atomic values.
- No repeating groups.

2NF

- 1NF
- No partial dependency on part of a composite key.

3NF

- 2NF
- No transitive dependency.

For your interviews, you should be able to explain **why normalization is useful**, not just recite the definitions.

🔥 Interview Questions You Should Be Able to Answer

Try answering these yourself:

1. What is the difference between DBMS and RDBMS?
2. What is a primary key?
3. Can a table have two primary keys?
4. Can a primary key contain NULL?
5. What is a foreign key?
6. Can a foreign key contain duplicate values?
7. What is a composite key?
8. Primary Key vs Unique Constraint?
9. What are database constraints?
10. Explain one-to-one, one-to-many, and many-to-many.
11. How do you implement many-to-many relationships?
12. What is normalization?
13. Explain 1NF, 2NF, and 3NF.
14. Why would you sometimes intentionally denormalize a database?

Mini challenge 🧠

Imagine you're designing a hospital database with:

⌕ Plain text

Doctor
Patient
Appointment

Tell me:

What would be the PK and FK in each table, and what relationships exist between them?

Then we'll move to **Group 2: SELECT + WHERE + ORDER BY + filtering**, with SQL interview questions and exercises.

Today 7:25 PM

ok let's move to the next group

Group 2 — SELECT, Filtering & Sorting

This group is basic, but interviewers use it to test whether you really understand SQL execution and filtering.

1. SELECT

Used to retrieve data.

</> SQL

```
SELECT Name, Salary  
FROM Employees;
```

Retrieve all columns:

</> SQL

```
SELECT *  
FROM Employees;
```

Interview question

Why should you generally avoid `SELECT *` in production?

Because it:

- Retrieves unnecessary columns.
- Transfers more data.
- Can increase I/O.
- Makes queries less explicit.
- Can cause problems if the table structure changes.

2. DISTINCT

Removes duplicate results.

</> SQL

```
SELECT DISTINCT DepartmentId  
FROM Employees;
```

If employees have:

</> Plain text

```
DepartmentId  
1  
1
```

2
2
3

Result:

</> Plain text

1
2
3

Important

`DISTINCT` applies to the **entire selected combination**.

</> SQL

```
SELECT DISTINCT DepartmentId, JobTitle
FROM Employees;
```

The combination of `DepartmentId` + `JobTitle` must be unique.

3. WHERE

Filters rows.

</> SQL

```
SELECT *
FROM Employees
WHERE Salary > 10000;
```

Only rows satisfying the condition are returned.

4. Comparison Operators

Know these:

</> Plain text

=	equal
<>	not equal
!=	not equal
>	greater
<	less
>=	greater than or equal
<=	less than or equal

Example:

</> SQL

```
SELECT *  
FROM Employees  
WHERE Salary >= 10000;
```

5. AND / OR / NOT

AND

Both conditions must be true.

</> SQL

```
WHERE Salary > 10000  
AND DepartmentId = 2
```

OR

At least one condition must be true.

</> SQL

```
WHERE DepartmentId = 1  
OR DepartmentId = 2
```

NOT

Negates a condition.

</> SQL

```
WHERE NOT DepartmentId = 2
```

⚠️ AND + OR precedence

This is an important interview trap.

Consider:

`</> SQL`

```
WHERE DepartmentId = 1
      OR DepartmentId = 2
      AND Salary > 10000
```

AND is evaluated before OR .

So it's effectively:

`</> SQL`

```
WHERE DepartmentId = 1
      OR (DepartmentId = 2 AND Salary > 10000)
```

If you mean:

Department 1 or 2, and salary > 10000

write:

`</> SQL`

```
WHERE (DepartmentId = 1 OR DepartmentId = 2)
      AND Salary > 10000
```

Use parentheses when the intended logic isn't obvious.

6. IN

Instead of:

`</> SQL`

```
WHERE DepartmentId = 1
      OR DepartmentId = 2
      OR DepartmentId = 3
```

you can write:

```
</> SQL
```

```
WHERE DepartmentId IN (1, 2, 3)
```

Very common.

7. NOT IN

```
</> SQL
```

```
WHERE DepartmentId NOT IN (1, 2, 3)
```

⚠ Important NULL issue

`NOT IN` can produce surprising results when the compared values involve `NULL`.

For example, if a subquery returns:

```
</> Plain text
```

```
1  
2  
NULL
```

then `NOT IN` may cause no rows to qualify because of SQL's three-valued logic.

This becomes particularly important with subqueries.

8. BETWEEN

Used for ranges.

```
</> SQL
```

```
WHERE Salary BETWEEN 10000 AND 20000
```

`BETWEEN` is **inclusive**.

So:

</> Plain text

10000 

15000 

20000 

9. LIKE

Used for pattern matching.

Starts with K

</> SQL

```
WHERE Name LIKE 'K%'
```

Ends with n

</> SQL

```
WHERE Name LIKE '%n'
```

Contains "a"

</> SQL

```
WHERE Name LIKE '%a%'
```

Exactly one unknown character

</> SQL

```
WHERE Name LIKE '_haled'
```

_ = one character

% = zero or more characters

10. NULL

This is very important.

NULL means:

The value is unknown / missing / not applicable.

It is **not**:

</> Plain text

```
0
''
false
```

You cannot correctly test NULL using:

</> SQL

```
WHERE ManagerId = NULL
```

Instead:

</> SQL

```
WHERE ManagerId IS NULL
```

or:

</> SQL

```
WHERE ManagerId IS NOT NULL
```

11. ORDER BY

Sorts the result.

Ascending:

</> SQL

```
SELECT *
FROM Employees
ORDER BY Salary ASC;
```

Descending:

</> SQL

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC;
```

ASC is the default.

So:

</> SQL

```
ORDER BY Salary
```

means:

</> SQL

```
ORDER BY Salary ASC
```

12. Multiple Columns in ORDER BY

You can sort using multiple columns:

</> SQL

```
SELECT *  
FROM Employees  
ORDER BY DepartmentId ASC, Salary DESC;
```

Meaning:

1. Sort by department ascending.
2. Within each department, sort salary descending.

13. TOP

SQL Server-specific.

```
</> SQL
```

```
SELECT TOP 5 *  
FROM Employees  
ORDER BY Salary DESC;
```

Gets the top 5 highest-paid employees.

Why ORDER BY matters

Without:

```
</> SQL
```

```
ORDER BY
```

you should **not assume** which 5 rows SQL Server will return.

14. TOP WITH TIES

Interesting SQL Server feature:

```
</> SQL
```

```
SELECT TOP 3 WITH TIES *  
FROM Employees  
ORDER BY Salary DESC;
```

Suppose salaries are:

```
</> Plain text
```

```
50000  
40000  
30000  
30000  
20000
```

TOP 3 WITH TIES can return:

```
</> Plain text
```

```
50000  
40000
```

```
30000
```

```
30000
```

because both employees tied at the third position.

15. OFFSET / FETCH

Useful for pagination.

`</>` SQL

```
SELECT *  
FROM Employees  
ORDER BY Id  
OFFSET 20 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

Meaning:

Skip the first 20 rows and return the next 10.

This is important for backend APIs.

For example:

`</>` Plain text

```
Page 1 → OFFSET 0  
Page 2 → OFFSET 10  
Page 3 → OFFSET 20
```

Logical Query Processing Order

This is **very important** for SQL interviews.

When you write:

`</>` SQL

```
SELECT ...  
FROM ...  
WHERE ...  
GROUP BY ...
```

```
HAVING ...  
ORDER BY ...
```

SQL logically processes it approximately as:

</> Plain text

```
FROM  
↓  
WHERE  
↓  
GROUP BY  
↓  
HAVING  
↓  
SELECT  
↓  
DISTINCT  
↓  
ORDER BY  
↓  
TOP / OFFSET-FETCH
```

This explains many SQL behaviors.

For example, why can't you normally use a `SELECT` alias inside `WHERE` ?

Because logically:

</> Plain text

```
WHERE
```

happens **before**

</> Plain text

```
SELECT
```

Interview Questions

Make sure you can answer these:

Q1

What's the difference between `WHERE` and `HAVING` ?

We'll cover `HAVING` deeply in the aggregation group, but the short answer:

- `WHERE` filters **rows before grouping**
 - `HAVING` filters **groups after grouping**
-

Q2

What's the difference between:

```
</> SQL
```

```
= NULL
```

and:

```
</> SQL
```

```
IS NULL
```

Answer: `NULL` represents an unknown value, so normal equality comparison doesn't work. Use `IS NULL` .

Q3

What's the difference between `IN` and `BETWEEN` ?

`IN` checks whether a value matches one of several specified values.

`BETWEEN` checks whether a value falls within an inclusive range.

Q4

What's the difference between `%` and `_` in `LIKE` ?

```
</> Plain text
```

```
% → zero or more characters
```

```
_ → exactly one character
```

Q5

What's the difference between `DISTINCT` and `GROUP BY` ?

They can sometimes produce similar results, but they serve different purposes:

`DISTINCT` → remove duplicate result combinations.

`GROUP BY` → create groups, usually for aggregation.



Practice

Using:

</> Plain text

Employees

Id

Name

Age

Salary

DepartmentId

HireDate

Try these without looking anything up:

1. Get all employees whose salary is greater than 15,000.
2. Get employees between 25 and 35 years old.
3. Get employees from departments 1, 3, or 5.
4. Get employees whose names start with A .
5. Get employees whose salary is between 10,000 and 20,000, sorted from highest salary to lowest.
6. Get the 5 highest-paid employees.
7. Get distinct department IDs.
8. Get employees who don't have a department.
9. Get the second page of employees assuming 10 employees per page.

After this, **Group 3 will be Aggregate Functions + GROUP BY + HAVING**, which is where SQL starts becoming much more interesting.

Group 3 — Aggregate Functions, GROUP BY & HAVING

This is one of the **most important SQL interview groups**. You should be comfortable writing these queries without thinking too much.

1. Aggregate Functions

Aggregate functions take **multiple rows** and produce a result.

The main ones:

Function	Purpose
COUNT()	Count rows/values
SUM()	Calculate total
AVG()	Calculate average
MIN()	Find minimum
MAX()	Find maximum

Assume:

</> Plain text

Employees

```
-----  
Id   Name    Salary  DeptId  
1    Ahmed    10000   1  
2    Khaled    15000   1  
3    Ali       20000   2  
4    Omar      12000   2  
5    Sara      18000   2
```

2. COUNT()

Count all rows

</> SQL

```
SELECT COUNT(*)  
FROM Employees;
```

Result:

</> Plain text

5

COUNT(column)

</> SQL

```
SELECT COUNT(DepartmentId)  
FROM Employees;
```

Important:

COUNT(column) does **not count NULL values**.

While:

</> SQL

```
COUNT(*)
```

counts rows.

Very common interview question

What's the difference between:

</> SQL

```
COUNT(*)
```

and

</> SQL

```
COUNT(ColumnName)
```

Answer:

- `COUNT(*)` → counts rows.
 - `COUNT(ColumnName)` → counts non-NULL values in that column.
-

3. SUM()

Calculate the total.

</> SQL

```
SELECT SUM(Salary)
FROM Employees;
```

Result:

</> Plain text

75000

You can combine it with `WHERE` :

</> SQL

```
SELECT SUM(Salary)
FROM Employees
WHERE DepartmentId = 1;
```

4. AVG()

</> SQL

```
SELECT AVG(Salary)
FROM Employees;
```

Important:

`AVG()` ignores `NULL` values.

For example:

</> Plain text

```
10000  
20000  
NULL
```

Average is:

</> Plain text

```
15000
```

not:

</> Plain text

```
10000
```

5. MIN() and MAX()

</> SQL

```
SELECT MIN(Salary)  
FROM Employees;
```

</> SQL

```
SELECT MAX(Salary)  
FROM Employees;
```

You can retrieve multiple aggregates together:

</> SQL

```
SELECT  
    COUNT(*) AS EmployeeCount,  
    SUM(Salary) AS TotalSalary,  
    AVG(Salary) AS AverageSalary,  
    MIN(Salary) AS MinimumSalary,  
    MAX(Salary) AS MaximumSalary  
FROM Employees;
```

6. GROUP BY ★★ ★

This is the key concept.

Without GROUP BY :

`</> SQL`

```
SELECT AVG(Salary)
FROM Employees;
```

You get **one average for the entire table**.

But suppose you want:

Average salary for each department.

Use:

`</> SQL`

```
SELECT DepartmentId, AVG(Salary)
FROM Employees
GROUP BY DepartmentId;
```

Conceptually:

`</> Plain text`

```
Employees
  ↓
GROUP BY DepartmentId
  ↓
Department 1 → employees
Department 2 → employees
Department 3 → employees
  ↓
AVG(Salary) for each group
```

7. GROUP BY Multiple Columns

You can group by more than one column:

`</> SQL`

```
SELECT DepartmentId, JobTitle, COUNT(*)  
FROM Employees  
GROUP BY DepartmentId, JobTitle;
```

SQL creates groups based on the **combination**:

</> Plain text

DepartmentId + JobTitle

8. Important GROUP BY Rule ★

If you use `GROUP BY`, every selected column must generally be either:

1. In the `GROUP BY`, or
2. Inside an aggregate function.

For example:

</> SQL

```
SELECT DepartmentId, Name, AVG(Salary)  
FROM Employees  
GROUP BY DepartmentId;
```

✗ Invalid.

Why?

Because SQL doesn't know **which Name** it should return for a department containing multiple employees.

Correct:

</> SQL

```
SELECT DepartmentId, AVG(Salary)  
FROM Employees  
GROUP BY DepartmentId;
```

Or:

</> SQL

```
SELECT DepartmentId, Name, Salary
FROM Employees
GROUP BY DepartmentId, Name, Salary;
```

9. WHERE + GROUP BY

You can filter rows **before** grouping.

Example:

Calculate the average salary per department, but only consider employees earning more than 10,000.

</> SQL

```
SELECT DepartmentId, AVG(Salary)
FROM Employees
WHERE Salary > 10000
GROUP BY DepartmentId;
```

Conceptually:

</> Plain text

```
All employees
    ↓
WHERE Salary > 10000
    ↓
Remaining employees
    ↓
GROUP BY DepartmentId
    ↓
AVG()
```

10. HAVING

HAVING filters **groups**.

Example:

Find departments with more than 5 employees.

SQL

```
SELECT DepartmentId, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY DepartmentId
HAVING COUNT(*) > 5;
```

This is something `WHERE` cannot directly do:

SQL

```
WHERE COUNT(*) > 5 -- ❌
```

because `COUNT()` is calculated after the rows are grouped.

11. WHERE vs HAVING ★★ ★

This is a classic interview question.

WHERE

Filters individual rows **before** grouping.

SQL

```
WHERE Salary > 10000
```

HAVING

Filters groups **after** grouping.

SQL

```
HAVING AVG(Salary) > 15000
```

Think:

Plain text

```
WHERE
↓
Filter rows
↓
GROUP BY
↓
```

Create groups



HAVING



Filter groups

12. WHERE + HAVING Together

Example:

Find departments where employees earning more than 10,000 have an average salary greater than 15,000.

</> SQL

```
SELECT DepartmentId, AVG(Salary) AS AvgSalary
FROM Employees
WHERE Salary > 10000
GROUP BY DepartmentId
HAVING AVG(Salary) > 15000;
```

Execution conceptually:

</> Plain text

```
All Employees
  ↓
WHERE Salary > 10000
  ↓
GROUP BY DepartmentId
  ↓
AVG(Salary)
  ↓
HAVING AVG(Salary) > 15000
  ↓
Result
```

13. GROUP BY with JOIN ★ ★ ★

This is where your real backend SQL starts.

Suppose:

</> Plain text

Departments

Id

Name

Employees

Id

Name

Salary

DepartmentId

Question:

Get each department's name and average employee salary.

</> SQL

```
SELECT
    d.Name,
    AVG(e.Salary) AS AverageSalary
FROM Departments d
JOIN Employees e
    ON d.Id = e.DepartmentId
GROUP BY d.Name;
```

This pattern is **extremely common**.

14. COUNT + LEFT JOIN ⚠

This is a very useful interview concept.

Suppose you want:

Show all departments, including departments that have zero employees.

You need:

</> SQL

```
SELECT
    d.Name,
    COUNT(e.Id) AS EmployeeCount
FROM Departments d
LEFT JOIN Employees e
```

```
ON d.Id = e.DepartmentId  
GROUP BY d.Name;
```

Why `COUNT(e.Id)` instead of `COUNT(*)` ?

Because with `LEFT JOIN` , a department with no employees still produces a row containing `NULL` employee columns.

`COUNT(*)` would count that joined row.

`COUNT(e.Id)` ignores the `NULL`.

This distinction is **very worth remembering**.

15. DISTINCT vs GROUP BY

These can sometimes produce similar results.

For example:

```
</> SQL
```

```
SELECT DISTINCT DepartmentId  
FROM Employees;
```

and:

```
</> SQL
```

```
SELECT DepartmentId  
FROM Employees  
GROUP BY DepartmentId;
```

can produce the same department list.

But their purposes are different:

DISTINCT

Remove duplicate result rows.

GROUP BY

Create groups, usually so you can perform aggregation.

16. COUNT(DISTINCT)

You can combine them:

</> SQL

```
SELECT COUNT(DISTINCT DepartmentId)
FROM Employees;
```

Meaning:

How many different departments have employees?

17. Aggregate Functions and NULL

Remember:

</> Plain text

```
COUNT(*)
```

counts rows.

But:

</> Plain text

```
COUNT(Column)
SUM(Column)
AVG(Column)
MIN(Column)
MAX(Column)
```

generally ignore NULL values.

Example:

</> Plain text

```
Salary
-----
10000
```

```
20000  
NULL
```

AVG(Salary) :

```
</> Plain text
```

```
15000
```

not 10000.

Interview Questions

You should be able to answer these:

Q1

What's the difference between `WHERE` and `HAVING` ?

Q2

Why can't you normally write:

```
</> SQL
```

```
WHERE COUNT(*) > 5
```

?

Q3

What's the difference between:

```
</> SQL
```

```
COUNT(*)
```

and:

```
</> SQL
```

```
COUNT(column)
```

?

Q4

What happens to NULL values when using `AVG()` ?

Q5

Why does SQL require non-aggregated selected columns to appear in `GROUP BY` ?

Q6

What's the difference between `DISTINCT` and `GROUP BY` ?

Q7

Why might `COUNT(e.Id)` be preferable to `COUNT(*)` with a `LEFT JOIN` ?



Practice Questions

Using:

`</>` Plain text

Employees

Id

Name

Salary

DepartmentId

Departments

Id

Name

Try these:

1. Find the total salary of all employees.
2. Find the average salary of employees.
3. Find the highest salary in the company.
4. Find the number of employees in each department.
5. Find the average salary for each department.
6. Find departments that have more than 5 employees.

7. Find departments whose average salary is greater than 15,000.
8. Find the department with the highest average salary.
9. Display every department and the number of employees, including departments with zero employees.
10. 🔥 Find the highest salary in each department.

That last question is especially important because it leads directly into **subqueries, CTEs, and window functions**.

Next: **Group 4 — JOINS**, where we'll go deeply into `INNER` , `LEFT` , `RIGHT` , `FULL` , `CROSS` , and **SELF JOIN**, including the common interview traps.

Today 9:22 PM

go

Group 4 — JOINS ★ ★ ★

This is probably the **most important SQL topic** for a .NET backend interview. If you understand JOINS deeply, many SQL queries become much easier.

SQL Server uses JOINS to combine rows from multiple tables based on a relationship/condition. The main logical JOIN types are `INNER` , `LEFT` , `RIGHT` , `FULL` , and `CROSS` . 🌐 [Microsoft Learn](#) +1

Let's use these tables:

⌄ Plain text

Departments

Id	Name
1	IT
2	HR
3	Sales
4	Finance

⌄ Plain text

Employees

Id	Name	DepartmentId
1	Ahmed	1
2	Khaled	1

3	Sara	2
4	Omar	5

Notice:

- Department 3 has **no employees**
- Department 4 has **no employees**
- Omar has `DepartmentId = 5` , which doesn't exist in `Departments` — assuming the FK constraint isn't enforcing this for our example.

1. INNER JOIN ★★ ★

Returns **only rows that have a match in both tables**. This is the default JOIN if you simply write

JOIN .  Microsoft Learn

</> SQL

```
SELECT
    e.Name,
    d.Name AS DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.Id;
```

Result:

</> Plain text

Ahmed	IT
Khaled	IT
Sara	HR

Omar disappears because:

</> Plain text

```
Employee.DepartmentId = 5
```

but there is no:

</> Plain text

```
Department.Id = 5
```

Think of INNER JOIN as:

Give me only the matches.

2. LEFT JOIN ★ ★ ★

A LEFT JOIN keeps **every row from the left table**, even if there is no matching row on the right.

Unmatched right-side columns become NULL .  Microsoft Learn

`</>` SQL

```
SELECT
    d.Name AS DepartmentName,
    e.Name AS EmployeeName
FROM Departments d
LEFT JOIN Employees e
    ON d.Id = e.DepartmentId;
```

Result conceptually:

`</>` Plain text

IT	Ahmed
IT	Khaled
HR	Sara
Sales	NULL
Finance	NULL

Notice:

Sales and Finance are still there.

That's the whole point.

Think:

Give me everything from the LEFT table, and whatever matches from the RIGHT.

3. RIGHT JOIN

The opposite of LEFT JOIN.

</> SQL

```
SELECT
    d.Name AS DepartmentName,
    e.Name AS EmployeeName
FROM Departments d
RIGHT JOIN Employees e
    ON d.Id = e.DepartmentId;
```

It keeps **every employee**, even if their department doesn't match.

Result:

</> Plain text

IT	Ahmed
IT	Khaled
HR	Sara
NULL	Omar

In practice, many developers prefer rewriting a `RIGHT JOIN` as a `LEFT JOIN` by switching the table order because it tends to be easier to read.

4. FULL OUTER JOIN

Keeps **everything** from both tables.

Matching rows are combined.

Unmatched rows from either side are kept with `NULL` on the other side. [🌐 Microsoft Learn](#)

</> SQL

```
SELECT
    d.Name AS DepartmentName,
    e.Name AS EmployeeName
FROM Departments d
FULL OUTER JOIN Employees e
    ON d.Id = e.DepartmentId;
```

Conceptually:

</> Plain text

IT	Ahmed
IT	Khaled

HR	Sara
Sales	NULL
Finance	NULL
NULL	Omar

So:

⟨/⟩ Plain text

INNER → matching only

LEFT → everything from left

RIGHT → everything from right

FULL → everything from both

5. CROSS JOIN

This one is different.

It creates **every possible combination** of rows from both tables. SQL Server describes this as the Cartesian product. [🌐 Microsoft Learn](#)

Suppose:

⟨/⟩ Plain text

Departments = 4 rows

Employees = 4 rows

Then:

⟨/⟩ SQL

```
SELECT *  
FROM Departments  
CROSS JOIN Employees;
```

produces:

⟨/⟩ Plain text

4 × 4 = 16 rows

No ON condition.

For example:

</> Plain text

IT	Ahmed
IT	Khaled
IT	Sara
IT	Omar
HR	Ahmed
HR	Khaled
HR	Sara
HR	Omar
...	

When is CROSS JOIN useful?

For things like:

Generate every combination of products and available sizes.

or:

Generate every combination of dates and stores.

But accidentally using it can create a **huge result set**.

6. SELF JOIN

A table can join to **itself**.

Imagine:

</> Plain text

Employees

Id	Name	ManagerId
1	Ahmed	NULL
2	Khaled	1
3	Sara	1
4	Omar	2

You want:

Employee name + manager name.

You can join `Employees` to itself:

`</>` SQL

```
SELECT
    e.Name AS Employee,
    m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m
    ON e.ManagerId = m.Id;
```

Result:

`</>` Plain text

Ahmed	NULL
Khaled	Ahmed
Sara	Ahmed
Omar	Khaled

This is called a **self-join**.

The important thing is the aliases:

`</>` Plain text

e → employee
m → manager

Even though both refer to the same table.

7. Joining More Than Two Tables

Very important for backend work.

Suppose:

`</>` Plain text

Customer
Order

OrderItem
Product

Relationships:

</> Plain text

Customer
↓
Order
↓
OrderItem
↓
Product

You can do:

</> SQL

```
SELECT
    c.Name AS Customer,
    o.Id AS OrderId,
    p.Name AS Product
FROM Customer c
JOIN Orders o
    ON c.Id = o.CustomerId
JOIN OrderItems oi
    ON o.Id = oi.OrderId
JOIN Products p
    ON oi.ProductId = p.Id;
```

The key idea:

You don't need to join every table directly to every other table. You can use the relationships between them to reach the data you need.

8. JOIN + WHERE

Example:

Get employees who work in IT.

</> SQL

```
SELECT
    e.Name,
    d.Name AS Department
FROM Employees e
JOIN Departments d
    ON e.DepartmentId = d.Id
WHERE d.Name = 'IT';
```

Think:

</> Plain text

```
JOIN
↓
Combine related rows
↓
WHERE
↓
Filter the result
```

9. The BIG LEFT JOIN Trap ★★ ★

This is one of the most important JOIN interview questions.

Suppose you want:

Show **all departments**, but only employees whose salary is greater than 10,000.

You might write:

</> SQL

```
SELECT
    d.Name,
    e.Name
FROM Departments d
LEFT JOIN Employees e
    ON d.Id = e.DepartmentId
WHERE e.Salary > 10000;
```

⚠ Be careful.

The `WHERE` condition eliminates rows where `e.Salary` is `NULL`.

That can effectively make your `LEFT JOIN` behave like an `INNER JOIN` for this condition.

Put the condition in ON

Instead:

</> SQL

```
SELECT
    d.Name,
    e.Name
FROM Departments d
LEFT JOIN Employees e
    ON d.Id = e.DepartmentId
AND e.Salary > 10000;
```

Now the department is preserved even if it has no employee satisfying the salary condition.

This distinction matters because predicates in `ON` and `WHERE` can produce different results with outer joins.  Microsoft Learn

10. ON vs WHERE

This is worth memorizing conceptually.

ON

Defines:

How should the tables be matched?

</> SQL

```
ON e.DepartmentId = d.Id
```

WHERE

Defines:

Which rows from the resulting query should I keep?

</> SQL

```
WHERE e.Salary > 10000
```

For `INNER JOIN`, moving many conditions between `ON` and `WHERE` can produce the same result.

For OUTER JOIN , it can change the result.  Microsoft Learn

11. JOIN Does NOT Necessarily Mean Foreign Key

A common misconception:

"Can I JOIN only PK with FK?"

No.

The JOIN condition can compare other compatible columns too.

For example:

```
</> SQL
```

```
ON e.Email = u.Email
```

Although in well-designed relational databases, PK/FK relationships are a very common basis for joins.

SQL Server's JOIN condition can use various logical predicates, not just equality.  Microsoft Learn

12. JOIN Can Produce Duplicate Rows

Suppose:

```
</> Plain text
```

```
Department
```

```
-----
```

```
IT
```

```
Employees
```

```
-----
```

```
Ahmed IT
```

```
Khaled IT
```

If you join:

```
</> SQL
```

```
Department
```

```
JOIN Employees
```

you get:

</> Plain text

IT	Ahmed
IT	Khaled

The department appears twice.

That's **not necessarily a problem**.

It happens because there are two matching employee rows.

This is especially important with **one-to-many relationships**.

13. JOIN + GROUP BY

Now combine what we learned in Group 3.

Find the number of employees in each department.

</> SQL

```
SELECT
    d.Name,
    COUNT(e.Id) AS EmployeeCount
FROM Departments d
LEFT JOIN Employees e
    ON d.Id = e.DepartmentId
GROUP BY d.Name;
```

This gives you:

</> Plain text

IT	2
HR	1
Sales	0
Finance	0

Notice again:

</> SQL

```
COUNT(e.Id)
```

rather than:

```
</> SQL
```

```
COUNT(*)
```

because we want departments with no matching employees to have count 0 .

14. JOIN vs UNION

Another common interview question.

JOIN

Combines **columns** from related tables.

```
</> Plain text
```

```
Table A + Table B
```

```
↓
```

```
More columns
```

UNION

Combines **rows** from compatible result sets.

```
</> Plain text
```

```
Query A
```

```
+
```

```
Query B
```

```
↓
```

```
More rows
```

So:

```
</> Plain text
```

```
JOIN → horizontally
```

```
UNION → vertically
```

Not literally how the engine internally operates, but it's a useful mental model.

15. Physical JOIN Algorithms — SQL Server ★ ★

You don't need to deeply optimize these yet, but know the names.

SQL Server can execute joins using algorithms such as:

- Nested Loops
- Merge Join
- Hash Join
- Adaptive Join

The optimizer chooses an execution strategy based on things such as table sizes, indexes, and data distribution.  Microsoft Learn

We'll cover these properly later when we reach **Execution Plans & Performance**.

Interview Questions

Make sure you can answer these:

1. What is a JOIN?
2. INNER JOIN vs LEFT JOIN?
3. LEFT JOIN vs RIGHT JOIN?
4. What does FULL OUTER JOIN return?
5. What is CROSS JOIN?
6. What is a SELF JOIN?
7. Can you JOIN more than two tables?
8. What's the difference between ON and WHERE ?
9. Why can a LEFT JOIN accidentally behave like an INNER JOIN ?
10. Why can a JOIN produce duplicate rows?
11. JOIN vs UNION?
12. What JOIN types does SQL Server support?
13. What is a Cartesian product?
14. What are Nested Loop, Hash, and Merge joins?

Practice — Do These Yourself

Using:

 Plain text

Customers

Id

Name

Orders

Id

CustomerId

OrderDate

TotalAmount

1.
Get every customer and their orders.
2.
Get only customers who have placed an order.
3.
Get all customers, including customers who have never placed an order.
4.
Find customers who have never placed an order.
5.
Get each customer's total number of orders.
6.
Get each customer's total order amount.
7. 🔥
Get the customer with the highest total order amount.
8. 🔥
Get all customers who have placed at least 3 orders.
9. 🔥
Suppose we add:

</> Plain text

OrderItems

Id

```
OrderId
ProductId
Quantity
```

and:

</> Plain text

```
Products
-----
Id
Name
Price
```

Write the conceptual JOIN chain to get:

</> Plain text

```
Customer → Order → OrderItem → Product
```

Don't worry about solving all of them perfectly yet. The important thing is that you can look at a database relationship diagram and immediately know **which JOIN you need and why**.

Next group: Subqueries + EXISTS + IN + correlated subqueries. This is where we'll start solving questions like *"find employees earning more than the company average"* and *"find the second-highest salary."*

go

Group 5 — Subqueries, IN, EXISTS & Correlated Subqueries



Subqueries are **queries inside another query**. In SQL Server, they can appear in `SELECT` , `WHERE` , `HAVING` , `INSERT` , `UPDATE` , `DELETE` , and even inside another subquery.  Microsoft Learn

We'll use:

</> Plain text

```
Employees
-----
Id
Name
Salary
DepartmentId
```

Departments

Id

Name

1. Basic Subquery

Suppose:

Find employees whose salary is greater than the average salary.

First, you could calculate:

`</>` SQL

```
SELECT AVG(Salary)
FROM Employees;
```

Then use that result:

`</>` SQL

```
SELECT *
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees
);
```

The inner query:

`</>` SQL

```
SELECT AVG(Salary)
FROM Employees
```

returns **one value**.

The outer query compares every employee's salary with that value.

Mental model

`</>` Plain text

Outer Query

↓

"Give me employees..."

↓

Inner Query

↓

"What's the average salary?"

↓

Outer Query continues

A scalar subquery used with `>`, `<`, `=`, etc. must return a single value; if it returns multiple rows, SQL Server raises an error. [Microsoft Learn](#)

2. Subquery with `IN` ★ ★ ★

`IN` is useful when the subquery returns **multiple values**.

Example:

Get employees who work in departments located in Cairo.

Suppose:

Plain text

Departments

Id	Name	City
1	IT	Cairo
2	HR	Giza
3	Sales	Cairo

Query:

SQL

```
SELECT *
FROM Employees
WHERE DepartmentId IN (
    SELECT Id
    FROM Departments
    WHERE City = 'Cairo'
);
```

The inner query might return:

</> Plain text

1
3

Then conceptually:

</> SQL

```
WHERE DepartmentId IN (1, 3)
```

So employees from departments 1 and 3 are returned.

3. IN vs =

This is important.

=

Expects a **single value**:

</> SQL

```
WHERE DepartmentId = (  
    SELECT Id  
    FROM Departments  
    WHERE Name = 'IT'  
);
```

This works if the subquery returns exactly one row.

IN

Can handle multiple values:

</> SQL

```
WHERE DepartmentId IN (  
    SELECT Id  
    FROM Departments  
    WHERE City = 'Cairo'  
);
```

If the inner query returns:

</> Plain text

1
3
5

IN is fine.

4. NOT IN ⚠️

Opposite idea:

</> SQL

```
SELECT *  
FROM Employees  
WHERE DepartmentId NOT IN (  
    SELECT Id  
    FROM Departments  
    WHERE City = 'Cairo'  
);
```

Meaning:

Give me employees whose department isn't one of the Cairo departments.

⚠️ NULL trap

Be careful when the subquery can return NULL .

Because SQL uses **three-valued logic** (TRUE , FALSE , UNKNOWN), NOT IN with a NULL in the subquery result can produce unexpected results.

This is one reason NOT EXISTS is often safer when checking non-existence.

5. EXISTS ★★ ★

EXISTS doesn't care about the actual values returned.

It asks:

Does the subquery return at least one row?

Example:

Get customers who have placed at least one order.

</> SQL

```
SELECT *  
FROM Customers c  
WHERE EXISTS (  
    SELECT *  
    FROM Orders o  
    WHERE o.CustomerId = c.Id  
);
```

The important part:

</> SQL

```
EXISTS (...)
```

returns logically:

</> Plain text

```
TRUE
```

if at least one matching row exists, otherwise:

</> Plain text

```
FALSE
```

SQL Server documents `EXISTS` specifically as an existence test. [🌐 Microsoft Learn](#)

6. EXISTS Doesn't Care What You SELECT

These are effectively the same existence test:

</> SQL

```
WHERE EXISTS (  
    SELECT *  
    FROM Orders o
```

```
WHERE o.CustomerId = c.Id
)
```

and:

`</>` SQL

```
WHERE EXISTS (
  SELECT o.Id
  FROM Orders o
  WHERE o.CustomerId = c.Id
)
```

and even:

`</>` SQL

```
WHERE EXISTS (
  SELECT 1
  FROM Orders o
  WHERE o.CustomerId = c.Id
)
```

Because you're asking:

Does at least one row exist?

You're not actually retrieving the selected value.

Microsoft's documentation notes that the select list of an `EXISTS` subquery doesn't determine the existence result. [🌐 Microsoft Learn](#)

7. NOT EXISTS

Opposite:

Get customers who have **never** placed an order.

`</>` SQL

```
SELECT *
FROM Customers c
WHERE NOT EXISTS (
  SELECT 1
  FROM Orders o
```

```
WHERE o.CustomerId = c.Id  
);
```

Think:

</> Plain text

EXISTS

→ at least one matching row

NOT EXISTS

→ zero matching rows

This is extremely common in real applications.

8. IN vs EXISTS

Classic interview question.

IN

Conceptually:

Is this value inside the list returned by the subquery?

</> SQL

```
WHERE DepartmentId IN (  
    SELECT Id  
    FROM Departments  
)
```

EXISTS

Conceptually:

Does a matching row exist?

</> SQL

```
WHERE EXISTS (  
    SELECT 1  
    FROM Departments d  
    WHERE d.Id = e.DepartmentId  
)
```

A major difference is that `EXISTS` is naturally suited to **correlated existence checks**.

Don't memorize:

"EXISTS is always faster than IN."

That's too simplistic.

SQL Server can optimize semantically equivalent queries, and Microsoft notes that subqueries and equivalent joins can sometimes have the same execution plan.  Microsoft Learn

So choose based on **what you're expressing**, then inspect the execution plan when performance actually matters.

9. Correlated Subquery

This is the important one.

A correlated subquery references a column from the **outer query**.

Example:

Find employees whose salary is greater than the average salary of their own department.

 SQL

```
SELECT *  
FROM Employees e  
WHERE e.Salary > (  
    SELECT AVG(e2.Salary)  
    FROM Employees e2  
    WHERE e2.DepartmentId = e.DepartmentId  
);
```

Look carefully:

 SQL

```
WHERE e2.DepartmentId = e.DepartmentId
```

`e.DepartmentId` belongs to the **outer query**.

That's what makes this correlated.

Conceptually:

</> Plain text

```
Employee Ahmed
  ↓
Find average salary of Ahmed's department
  ↓
Compare Ahmed's salary

Employee Khaled
  ↓
Find average salary of Khaled's department
  ↓
Compare Khaled's salary

...
```

The subquery depends on the current outer row. Microsoft describes correlated subqueries as subqueries that depend on values from the outer query and therefore conceptually repeat for the relevant outer rows. [Microsoft Learn](#)

10. Normal vs Correlated Subquery

Non-correlated

</> SQL

```
SELECT *
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary)
    FROM Employees
);
```

The inner query doesn't reference the outer query.

</> Plain text

Outer → Inner

Independent

Correlated

</> SQL

```
SELECT *  
FROM Employees e  
WHERE Salary > (  
    SELECT AVG(Salary)  
    FROM Employees e2  
    WHERE e2.DepartmentId = e.DepartmentId  
);
```

The inner query references:

</> Plain text

e.DepartmentId

from the outer query.

</> Plain text

Outer ↔ Inner

11. Subquery in SELECT

A subquery doesn't have to be in `WHERE` .

Example:

Display each employee and the company's average salary.

</> SQL

```
SELECT  
    Name,  
    Salary,  
    (SELECT AVG(Salary) FROM Employees) AS CompanyAverage  
FROM Employees;
```

Conceptually:

</> Plain text

Ahmed	10000	15000
Khaled	20000	15000
Sara	15000	15000

SQL Server allows subqueries to be used where an expression can be used.  Microsoft Learn

12. Subquery vs JOIN

Many problems can be solved using either.

Subquery

 SQL

```
SELECT *  
FROM Employees  
WHERE DepartmentId IN (  
    SELECT Id  
    FROM Departments  
    WHERE Name = 'IT'  
);
```

JOIN

 SQL

```
SELECT e.*  
FROM Employees e  
JOIN Departments d  
    ON e.DepartmentId = d.Id  
WHERE d.Name = 'IT';
```

Both can return the same result.

Which is better?

Don't use a blanket rule.

Think about:

- Readability
- Whether you're checking existence
- Whether you need columns from the other table
- Actual execution plan

- Data size

SQL Server's optimizer can produce equivalent plans for semantically equivalent approaches.



13. A Very Important Problem

Find the second-highest salary

One way:

`</> SQL`

```
SELECT MAX(Salary)
FROM Employees
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Employees
);
```

Think about it:

`</> Plain text`

1. Find maximum salary
↓
2. Remove that salary
↓
3. Find maximum of what's left
↓
4. That's the second-highest salary

But be careful with **duplicate salaries** and with the distinction between:

second-highest **distinct** salary

and

salary of the second employee after sorting.

We'll revisit this when we reach **window functions**, where `DENSE_RANK()` gives a cleaner solution for many ranking problems.

14. ANY / ALL — Know the Concept

These are less common in everyday backend code but can appear in interviews.

Suppose:

</> Plain text

10

20

30

> **ALL**

</> Plain text

> ALL (10, 20, 30)

means:

Greater than **every** value.

So effectively:

</> Plain text

> 30

> **ANY**

means:

Greater than **at least one** value.

So:

</> Plain text

> ANY (10, 20, 30)

effectively means:

</> Plain text

> 10

SQL Server supports **ANY** , **SOME** , and **ALL** with comparison operators.  Microsoft Learn

For your current interview preparation, understand the concept but don't spend too much time here.

The 5 Things You MUST Remember

1. Scalar subquery

Returns **one value**:

`</> SQL`

```
WHERE Salary > (  
    SELECT AVG(Salary)  
    FROM Employees  
)
```

2. IN

Checks a value against a **list**:

`</> SQL`

```
WHERE DepartmentId IN (...)
```

3. EXISTS

Checks whether **at least one matching row exists**:

`</> SQL`

```
WHERE EXISTS (...)
```

4. NOT EXISTS

Checks that **no matching row exists**:

`</> SQL`

```
WHERE NOT EXISTS (...)
```

5. Correlated subquery

Inner query references the outer query:

`</> SQL`

```
WHERE e2.DepartmentId = e.DepartmentId
```

Interview Questions

Try to answer these:

1. What is a subquery?
2. Where can a subquery be used?
3. What's the difference between a scalar subquery and a multi-row subquery?
4. IN vs EXISTS ?
5. NOT IN vs NOT EXISTS ?
6. What is a correlated subquery?
7. How do you find employees earning more than the company average?
8. How do you find employees earning more than their department's average?
9. How do you find customers who have never placed an order?
10. Can a subquery be replaced by a JOIN?
11. What happens if a scalar subquery returns multiple rows?
12. What do ANY and ALL mean?

Practice

Using:

</> Plain text

Employees

Id

Name

Salary

DepartmentId

Departments

Id

Name

Customers

Id

Name

Orders

Try these:

1.

Find employees whose salary is greater than the **company average**.

2.

Find employees whose salary is greater than the **average salary of their department**.

3.

Find employees who work in the **IT department** using `IN` .

4.

Find employees who work in the **IT department** using `EXISTS` .

5.

Find customers who have placed at least one order using `EXISTS` .

6.

Find customers who have never placed an order using `NOT EXISTS` .

7. 🔥

Find the second-highest distinct salary.

8. 🔥

Find departments that have **at least one employee earning more than 20,000**.

9. 🔥

Find employees whose salary is greater than **every employee in department 2**.

10.

Explain whether this is correlated or non-correlated:

SQL

```
SELECT *  
FROM Employees e  
WHERE e.Salary > (  
    SELECT AVG(Salary)  
    FROM Employees  
);
```

Once you're comfortable with these, **Group 6 is CTEs + Recursive CTEs**, followed by **Window Functions**. Those two topics are where you'll start solving much more advanced SQL interview problems.

go

Group 6 — CTEs (Common Table Expressions) ★ ★ ★

CTEs are very useful for writing **complex SQL queries in a clean, readable way**. They are especially common in interview questions involving multiple steps, ranking, recursion, and hierarchical data.

A CTE is a named result set defined with `WITH` and available to the **single statement immediately following it**. It isn't automatically materialized as a temporary table.  Microsoft Learn

1. What is a CTE?

Instead of writing a complicated subquery:

</> SQL

```
SELECT *
FROM (
    SELECT DepartmentId, AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
) x
WHERE AvgSalary > 15000;
```

You can write:

</> SQL

```
WITH DepartmentSalary AS
(
    SELECT
        DepartmentId,
        AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
)
SELECT *
FROM DepartmentSalary
WHERE AvgSalary > 15000;
```

Much easier to read.

Think of it as:

</> Plain text

WITH

↓

Create a temporary named result

↓

Use that result in the next statement

2. Basic CTE Syntax

The general structure is:

</> SQL

```
WITH CTE_Name AS
(
    SELECT ...
)
SELECT ...
FROM CTE_Name;
```

Example:

</> SQL

```
WITH HighEarnings AS
(
    SELECT *
    FROM Employees
    WHERE Salary > 20000
)
SELECT *
FROM HighEarnings;
```

The CTE itself isn't a permanent database object.

After the statement finishes, the CTE is gone.

3. CTE vs Temporary Table

This is an important interview question.

CTE

</> Plain text

```
WITH ...  
SELECT ...
```

- Exists for one statement.
- Mainly improves query organization/readability.
- Not automatically materialized.
- Useful for breaking complex queries into logical steps.

Temporary table

</> SQL

```
CREATE TABLE #Employees (...)
```

- Exists in the session/scope according to SQL Server's temp-table rules.
- Can be referenced by multiple statements.
- Can have indexes.
- Useful when you need to reuse an intermediate result.

Microsoft specifically notes that CTE results aren't materialized; if you need multiple references to the same result, a temporary object may be more appropriate. [🌐 Microsoft Learn](#)

Interview answer:

A CTE is primarily a query expression used within one statement, while a temporary table stores an intermediate result that can be reused across statements.

4. CTE vs Subquery

Suppose:

Find departments whose average salary exceeds 15,000.

Subquery

</> SQL

```
SELECT *  
FROM
```

```
(
    SELECT
        DepartmentId,
        AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
) d
WHERE AvgSalary > 15000;
```

CTE

```
</> SQL

WITH DepartmentSalary AS
(
    SELECT
        DepartmentId,
        AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
)
SELECT *
FROM DepartmentSalary
WHERE AvgSalary > 15000;
```

Both can solve the same problem.

The CTE version is often easier to read when the intermediate query becomes complicated.

5. Multiple CTEs ★ ★

You can define multiple CTEs in the same `WITH` clause:

```
</> SQL

WITH
DepartmentSalary AS
(
    SELECT
        DepartmentId,
        AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
),
HighSalaryDepartments AS
(
    SELECT *
```

```
FROM DepartmentSalary
WHERE AvgSalary > 15000
)
SELECT *
FROM HighSalaryDepartments;
```

Notice:

</> Plain text

```
DepartmentSalary
    ↓
HighSalaryDepartments
    ↓
Final SELECT
```

A later CTE can reference an earlier CTE in the same `WITH` clause. [Microsoft Learn](#)

6. CTE with JOIN

Very common:

</> SQL

```
WITH EmployeeData AS
(
    SELECT
        e.Name AS EmployeeName,
        e.Salary,
        d.Name AS DepartmentName
    FROM Employees e
    JOIN Departments d
        ON e.DepartmentId = d.Id
)
SELECT *
FROM EmployeeData
WHERE Salary > 15000;
```

The CTE essentially gives you a clean intermediate dataset.

7. CTE with GROUP BY

Example:

Find departments with more than 5 employees.

`</>` SQL

```
WITH DepartmentCounts AS
(
    SELECT
        DepartmentId,
        COUNT(*) AS EmployeeCount
    FROM Employees
    GROUP BY DepartmentId
)
SELECT *
FROM DepartmentCounts
WHERE EmployeeCount > 5;
```

You could use `HAVING` directly, but the CTE becomes useful when you want to perform **additional processing on the aggregated result**.

8. CTE for Multi-Step Problems

This is where CTEs become really useful.

Question:

Find the department with the highest average salary.

First:

`</>` SQL

```
WITH DepartmentSalary AS
(
    SELECT
        DepartmentId,
        AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
)
SELECT TOP 1 *
FROM DepartmentSalary
ORDER BY AvgSalary DESC;
```

Logical process:

</> Plain text

```
Employees
  ↓
GROUP BY Department
  ↓
Calculate average salary
  ↓
Name the result DepartmentSalary
  ↓
Sort by average salary
  ↓
TOP 1
```

9. CTE for Second Highest Salary

Another approach to a classic interview problem:

</> SQL

```
WITH Salaries AS
(
    SELECT DISTINCT Salary
    FROM Employees
)
SELECT MAX(Salary)
FROM Salaries
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Salaries
);
```

But later we'll see that **window functions** make ranking problems much more elegant.

10. Recursive CTE

This is the advanced part.

A recursive CTE references itself.

It's useful for hierarchical data such as:

</> Plain text

```
CEO
├── Manager A
│   ├── Employee 1
│   └── Employee 2
└── Manager B
    └── Employee 3
```

Imagine:

</> Plain text

Employees

Id

Name

ManagerId

Example:

</> Plain text

Id	Name	ManagerId
1	Ahmed	NULL
2	Khaled	1
3	Sara	1
4	Omar	2
5	Ali	2

You want to retrieve the entire hierarchy.

11. Structure of Recursive CTE

A recursive CTE has two important parts:

Anchor member

Gets the starting rows.

Recursive member

Gets the next level based on the previous result.

Conceptually:

</> Plain text

```
Anchor
↓
Level 1
↓
Recursive step
↓
Level 2
↓
Recursive step
↓
Level 3
↓
...
```

SQL Server requires the recursive CTE to contain an anchor member and a recursive member.

 Microsoft Learn

Example

</> SQL

```
WITH EmployeeHierarchy AS
(
    -- Anchor
    SELECT
        Id,
        Name,
        ManagerId,
        0 AS Level
    FROM Employees
    WHERE ManagerId IS NULL

    UNION ALL

    -- Recursive
    SELECT
        e.Id,
        e.Name,
        e.ManagerId,
        h.Level + 1
    FROM Employees e
```

Result conceptually:

</> Plain text

Id	Name	ManagerId	Level
1	Ahmed	NULL	0
2	Khaled	1	1
3	Sara	1	1
4	Omar	2	2
5	Ali	2	2

12. How Recursive CTE Executes

Suppose:

</> Plain text

```
Ahmed
├─ Khaled
│   └─ Omar
│       └─ Ali
└─ Sara
```

First:

</> Plain text

```
Anchor
↓
Ahmed
```

Then:

</> Plain text

```
Recursive step
↓
Khaled
Sara
```

Then:

</> Plain text

```
Recursive step
↓
Omar
Ali
```

Then there are no more employees below them, so recursion stops.

This is essentially how SQL Server evaluates recursive CTEs: anchor results are produced first, then the recursive member repeatedly consumes the previous level until it returns no rows. [Microsoft Learn](#) +1

13. MAXRECURSION ⚠

A badly designed recursive CTE can create an infinite loop.

SQL Server has `MAXRECURSION` to limit recursion depth.

Example:

SQL

```
WITH EmployeeHierarchy AS
(
    ...
)
SELECT *
FROM EmployeeHierarchy
OPTION (MAXRECURSION 10);
```

This limits the recursion to 10 levels.

SQL Server's default maximum recursion level for a statement is **100**; `MAXRECURSION 0` removes the limit, so it should be used carefully. [Microsoft Learn](#) +1

14. Recursive CTE Use Cases

You should remember these:

Organizational hierarchy

Plain text

```
CEO
↓
Manager
↓
Employee
```

Categories

</> Plain text

```

Electronics
├── Phones
│   ├── Android
│   └── iPhone
└── Laptops
  
```

File/folder structures**</> Plain text**

```

Root
├── Folder A
│   └── File
└── Folder B
  
```

Bill of Materials**</> Plain text**

```

Car
├── Engine
│   ├── Piston
│   └── Valve
└── Wheel
  
```

15. CTE Important Restrictions

For a normal CTE, remember:

- It must be followed by one statement that uses it.
- It isn't a permanent object.
- It isn't automatically materialized.
- You can define multiple CTEs in one `WITH` .
- In SQL Server, if the preceding statement in the batch isn't terminated, starting with `;WITH` is a common safe pattern. [🌐 Microsoft Learn](#)

For recursive CTEs, SQL Server imposes additional restrictions on the recursive member, including restrictions around `GROUP BY` , `HAVING` , `DISTINCT` , and outer joins. [🌐 Microsoft Learn](#)

Interview Questions

You should be able to answer:

1. What is a CTE?
2. Why would you use a CTE?
3. CTE vs subquery?
4. CTE vs temporary table?
5. Is a CTE a physical table?
6. Does a CTE store data permanently?
7. Can you have multiple CTEs?
8. Can one CTE reference another CTE?
9. What is a recursive CTE?
10. What are the anchor and recursive members?
11. What is `MAXRECURSION` ?
12. Where would you use a recursive CTE?
13. Can a CTE be used with `INSERT` , `UPDATE` , or `DELETE` ?
14. Does using a CTE automatically improve performance?

Important answer to #14:

No.

A CTE is primarily a query-organization mechanism. It doesn't automatically make a query faster. SQL Server can optimize the underlying query, and the CTE isn't automatically materialized.  Microsoft Learn



Practice

Using:

`</>` Plain text

Employees

Id

Name

Salary

DepartmentId

ManagerId

Try:

1.

Create a CTE containing employees whose salary is greater than `15000` , then select from it.

2.

Create a CTE calculating the average salary of each department, then return departments whose average is greater than 20000 .

3.

Use two CTEs:

- First → calculate employee count per department.
- Second → keep departments with more than 5 employees.

4. 🔥

Use a CTE to find the department with the highest average salary.

5. 🔥

Use a recursive CTE to display the employee-management hierarchy.

6. 🔥

Explain the difference between:

⌕ Plain text

CTE

Subquery

Temporary Table

View

🔔 One Important Correction to Your Mental Model

Don't think:

CTE = temporary table

That's the most common misunderstanding.

Think:

CTE = a named query expression that makes a complex statement easier to structure.

It has a temporary **scope**, but that doesn't mean SQL Server automatically creates a physical temporary table for it. [🌐 Microsoft Learn](#)

Next: Group 7 — Window Functions ★ ★ ★

This is a **very important interview group**:

- OVER()
- PARTITION BY
- ROW_NUMBER()
- RANK()
- DENSE_RANK()
- LAG()
- LEAD()
- Running totals
- Top-N per group
- Second/third highest salary
- Window functions vs GROUP BY

This is where you'll learn to solve many of the harder SQL interview questions cleanly.

go

Group 7 — Window Functions

This is **very important** for SQL interviews, especially `ROW_NUMBER` , `RANK` , `DENSE_RANK` , `LAG` , and `LEAD` .

1. What is a Window Function?

A **window function** calculates something across a set of related rows **without combining those rows into one row**.

The key syntax is:

```
</> SQL
```

```
FUNCTION(...) OVER (...)
```

For example:

```
</> SQL
```

```
SELECT
  Name,
  Salary,
  AVG(Salary) OVER () AS CompanyAverage
FROM Employees;
```

Every employee remains in the result, but each row also gets the company average.

That's the major difference from `GROUP BY` .

Microsoft describes `OVER` as defining the rows/partition on which the window function operates.

 Microsoft Learn

2. `OVER()`

Think of:

`</> SQL`

`OVER()`

as:

"Calculate this function using the entire result set."

Example:

`</> SQL`

```
SELECT
    Name,
    Salary,
    AVG(Salary) OVER () AS AvgSalary
FROM Employees;
```

If salaries are:

Name	Salary	AvgSalary
Ahmed	5000	7000
Ali	7000	7000
Omar	9000	7000

Unlike `GROUP BY` , you didn't lose the individual employees.

3. `PARTITION BY`

This is extremely important.

</> SQL

```
AVG(Salary) OVER (PARTITION BY DepartmentId)
```

means:

Calculate the average separately for each department.

Example:

</> SQL

```
SELECT
    Name,
    DepartmentId,
    Salary,
    AVG(Salary) OVER (
        PARTITION BY DepartmentId
    ) AS DepartmentAverage
FROM Employees;
```

Result conceptually:

Name	Department	Salary	DepartmentAverage
Ahmed	1	5000	6000
Ali	1	7000	6000
Omar	2	8000	9000
Hassan	2	10000	9000

Remember:

PARTITION BY $\neq$ GROUP BY

PARTITION BY **doesn't collapse rows.**

4. ORDER BY inside OVER

You can control the order in which the window function operates:

</> SQL

```
ROW_NUMBER() OVER (  
    ORDER BY Salary DESC  
)
```

This gives employees a ranking based on salary.

Important distinction:

```
</> SQL  
  
ORDER BY Salary DESC
```

at the end of the query controls the **final result order**.

But:

```
</> SQL  
  
OVER (ORDER BY Salary DESC)
```

controls the **window calculation**.

5. ROW_NUMBER()

Gives every row a **unique sequential number**.

```
</> SQL  
  
SELECT  
    Name,  
    Salary,  
    ROW_NUMBER() OVER (  
        ORDER BY Salary DESC  
    ) AS RowNum  
FROM Employees;
```

Example:

Name	Salary	RowNum
Ahmed	10000	1
Ali	9000	2

Name	Salary	RowNum
Omar	9000	3
Hassan	7000	4

Even if salaries are equal, the row numbers are different.

`ROW_NUMBER` requires an ordering and can also restart numbering within each `PARTITION BY`.

 Microsoft Learn

6. RANK()

`RANK()` gives the same rank to ties.

```
</> SQL
```

```
RANK() OVER (  
    ORDER BY Salary DESC  
)
```

Result:

Salary	RANK
10000	1
9000	2
9000	2
7000	4

Notice:

1, 2, 2, 4

There is a gap after the tie.  Microsoft Learn

7. DENSE_RANK()

Same idea, but **no gaps**.

</> SQL

```
DENSE_RANK() OVER (  
    ORDER BY Salary DESC  
)
```

Result:

Salary	DENSE_RANK
10000	1
9000	2
9000	2
7000	3

So:

ROW_NUMBER

</> Plain text

1, 2, 3, 4

RANK

</> Plain text

1, 2, 2, 4

DENSE_RANK

</> Plain text

1, 2, 2, 3

This difference is a **very common interview question**. SQL Server provides all three as ranking functions. [🌐 Microsoft Learn](https://learn.microsoft.com/en-us/sql/t-sql/functions/dense-rank-transact-sql?view=sql-server-2017)

8. The Most Important Use: Top N per Group

Suppose you want:

Get the top 2 highest-paid employees from **each department**.

First:

`</>` SQL

```
ROW_NUMBER() OVER (  
    PARTITION BY DepartmentId  
    ORDER BY Salary DESC  
)
```

Then put that result into a CTE/subquery and filter:

`</>` SQL

```
WHERE RowNum <= 2
```

Conceptually:

`</>` Plain text

```
Employees  
  ↓  
PARTITION BY Department  
  ↓  
ORDER BY Salary DESC  
  ↓  
ROW_NUMBER  
  ↓  
RowNum <= 2
```

This is one of the most useful real-world applications of window functions.

9. Second Highest Salary

Another classic interview question.

You can use:

```
</> SQL
```

```
DENSE_RANK() OVER (  
    ORDER BY Salary DESC  
)
```

Then:

```
</> SQL
```

```
WHERE Rank = 2
```

Why DENSE_RANK ?

Suppose:

```
</> Plain text
```

```
10000  
9000  
9000  
8000
```

The second-highest **distinct salary** is 9000 .

DENSE_RANK gives:

```
</> Plain text
```

```
10000 → 1  
9000 → 2  
9000 → 2  
8000 → 3
```

So both employees earning 9000 are returned.

10. LAG()

LAG() lets you access a value from a **previous row**.

Example:

```
</> SQL
```

```
SELECT
    OrderDate,
    Amount,
    LAG(Amount) OVER (
        ORDER BY OrderDate
    ) AS PreviousAmount
FROM Orders;
```

Result:

Date	Amount	PreviousAmount
Jan 1	100	NULL
Jan 2	150	100
Jan 3	120	150

Useful for:

- Comparing today's sales with yesterday's
- Salary changes
- Previous order
- Previous transaction
- Detecting changes

11. LEAD()

Opposite of LAG .

LEAD() gets a value from a **following** row.

</> SQL

```
LEAD(Amount) OVER (
    ORDER BY OrderDate
)
```

Result:

Date	Amount	NextAmount
Jan 1	100	150

Date	Amount	NextAmount
Jan 2	150	120
Jan 3	120	NULL

So:

Plain text

LAG → previous

LEAD → next

LEAD uses the ordering defined in its OVER clause to determine the following row. [Microsoft Learn](#)

12. Running Total

Window functions are excellent for cumulative calculations.

SQL

```
SELECT
    OrderDate,
    Amount,
    SUM(Amount) OVER (
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotal
FROM Orders;
```

Result:

Date	Amount	RunningTotal
Jan 1	100	100
Jan 2	200	300
Jan 3	150	450
Jan 4	300	750

Think:

</> Plain text

```
100
100 + 200
100 + 200 + 150
100 + 200 + 150 + 300
```

ROWS / RANGE can define the exact window frame used by the calculation.  Microsoft Learn

13. Window Aggregate Functions

You can use normal aggregate functions as window functions:

</> SQL

```
SUM() OVER()
AVG() OVER()
COUNT() OVER()
MIN() OVER()
MAX() OVER()
```

For example:

</> SQL

```
SELECT
    Name,
    DepartmentId,
    Salary,
    COUNT(*) OVER (
        PARTITION BY DepartmentId
    ) AS EmployeesInDepartment
FROM Employees;
```

Every employee gets the number of employees in their department.

14. Window Function vs GROUP BY

GROUP BY**</> SQL**

```
SELECT
    DepartmentId,
    AVG(Salary)
FROM Employees
GROUP BY DepartmentId;
```

Result:

</> Plain text

Department	Average
1	6000
2	9000

Employees disappeared into groups.

Window function

</> SQL

```
SELECT
    Name,
    DepartmentId,
    Salary,
    AVG(Salary) OVER (
        PARTITION BY DepartmentId
    )
FROM Employees;
```

Result:

</> Plain text

Ahmed	1	5000	6000
Ali	1	7000	6000
Omar	2	8000	9000
Hassan	2	10000	9000

Interview answer:

GROUP BY combines rows into groups, while window functions calculate values across related rows while keeping the individual rows.

15. ROW_NUMBER for Removing Duplicates

Very common practical problem.

Suppose you have duplicate customers.

You can assign:

</> SQL

```
ROW_NUMBER() OVER (  
    PARTITION BY Email  
    ORDER BY Id  
)
```

Then:

</> Plain text

Email	RowNumber
a@gmail.com	1
a@gmail.com	2
a@gmail.com	3
b@gmail.com	1

You can keep `RowNumber = 1` and identify/delete the duplicates.

★ Interview Questions You MUST Know

Q1. What is a window function?

A function that performs calculations across a set of related rows without collapsing them into a single row.

Q2. What does `OVER()` do?

Defines the window of rows on which the function operates.

Q3. What does `PARTITION BY` do?

Divides the result into groups/partitions, and the window calculation is performed separately for each partition.

Q4. Difference between ROW_NUMBER , RANK , and DENSE_RANK ?

</> Plain text

ROW_NUMBER → unique numbers
RANK → ties + gaps
DENSE_RANK → ties + no gaps

Q5. LAG vs LEAD ?

</> Plain text

LAG → previous row
LEAD → next row

Q6. Why use window functions instead of GROUP BY?

When you need aggregate/ranking information while still keeping every individual row.

Q7. How do you get top 3 employees from every department?

ROW_NUMBER() or RANK() / DENSE_RANK() with PARTITION BY DepartmentId , then filter the generated rank in an outer query/CTE.

Practice

Try these without looking at the answer:

1. Display every employee with their department's average salary.
2. Give every employee a salary rank within their department.
3. Find the top 3 highest-paid employees in each department.
4. Find the second-highest distinct salary.
5. Display each order with the previous order's amount.
6. Calculate a running total of sales ordered by date.
7. Find duplicate customers based on email using ROW_NUMBER().

Once these are comfortable, **Group 8 is CRUD + DDL/DML/TCL**, which is more straightforward.

Group 8 — CRUD + DDL / DML / TCL

This group is mostly about **changing database structure, changing data, and controlling transactions**.

1. The Big Picture

Think of SQL commands in categories:

Category	Purpose	Examples
DDL	Define/change database structure	CREATE , ALTER , DROP , TRUNCATE
DML	Work with data	SELECT , INSERT , UPDATE , DELETE
TCL	Control transactions	COMMIT , ROLLBACK , SAVE TRANSACTION
DCL	Control permissions	GRANT , DENY , REVOKE

Microsoft classifies CREATE , ALTER , DROP , and TRUNCATE as DDL, while SELECT , INSERT , UPDATE , and DELETE are DML. [🌐 Microsoft Learn](#)

2. CRUD

CRUD stands for:

🔗 Plain text

C → Create
R → Read
U → Update
D → Delete

In SQL:

🔗 Plain text

Create → INSERT
Read → SELECT

Update → UPDATE

Delete → DELETE

Example:

</> SQL

```
INSERT INTO Employees (Name, Salary)
VALUES ('Ahmed', 5000);
```

</> SQL

```
SELECT *
FROM Employees;
```

</> SQL

```
UPDATE Employees
SET Salary = 6000
WHERE Id = 1;
```

</> SQL

```
DELETE FROM Employees
WHERE Id = 1;
```

3. INSERT

Insert one row:

</> SQL

```
INSERT INTO Employees (Name, Salary, DepartmentId)
VALUES ('Ahmed', 5000, 2);
```

Multiple rows:

</> SQL

```
INSERT INTO Employees (Name, Salary)
VALUES
    ('Ahmed', 5000),
```

```
('Ali', 6000),  
('Omar', 7000);
```

You can also insert from another query:

</> SQL

```
INSERT INTO EmployeesBackup (Name, Salary)  
SELECT Name, Salary  
FROM Employees;
```

Interview tip

Always prefer specifying the column names:

</> SQL

```
INSERT INTO Employees (Name, Salary)  
VALUES (...);
```

rather than relying on the table's column order.

4. UPDATE

Basic:

</> SQL

```
UPDATE Employees  
SET Salary = 7000  
WHERE Id = 5;
```

Multiple columns:

</> SQL

```
UPDATE Employees  
SET  
    Salary = 7000,  
    DepartmentId = 3  
WHERE Id = 5;
```



Most important warning

If you forget `WHERE` :

`</> SQL`

```
UPDATE Employees  
SET Salary = 7000;
```

you update **every** employee.

Same danger exists with `DELETE` .

5. DELETE

Delete specific rows:

`</> SQL`

```
DELETE FROM Employees  
WHERE Id = 5;
```

Delete all rows:

`</> SQL`

```
DELETE FROM Employees;
```

But the table itself still exists.

6. DELETE vs TRUNCATE vs DROP

🔥 Very common interview question.

DELETE

`</> SQL`

```
DELETE FROM Employees  
WHERE DepartmentId = 5;
```

- DML
- Can use `WHERE`

- Deletes selected rows
 - Table remains
 - Can be rolled back when executed within an appropriate transaction
-

TRUNCATE

</> SQL

```
TRUNCATE TABLE Employees;
```

- DDL in SQL Server
- Removes **all rows**
- Cannot use `WHERE`
- Table structure remains
- Resets the identity counter in the usual SQL Server case
- Typically much more efficient than deleting every row individually

Microsoft lists `TRUNCATE TABLE` under DDL. [🌐 Microsoft Learn](#)

DROP

</> SQL

```
DROP TABLE Employees;
```

This removes the **table itself**.

So:

</> Plain text

DELETE

↓

remove rows

TRUNCATE

↓

remove all rows

DROP



remove table

Easy interview answer:

DELETE removes rows, TRUNCATE removes all rows while keeping the table structure, and DROP removes the table itself.

7. CREATE

Create a table:

</> SQL

```
CREATE TABLE Employees
(
    Id INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Salary DECIMAL(10,2),
    DepartmentId INT
);
```

You can create many other database objects:

</> Plain text

```
CREATE DATABASE
CREATE TABLE
CREATE VIEW
CREATE PROCEDURE
CREATE FUNCTION
CREATE INDEX
...
```

8. ALTER

Used to modify an existing object.

Add a column:

</> SQL

```
ALTER TABLE Employees  
ADD Email VARCHAR(200);
```

Change a column definition:

SQL

```
ALTER TABLE Employees  
ALTER COLUMN Name VARCHAR(150) NOT NULL;
```

Add a constraint:

SQL

```
ALTER TABLE Employees  
ADD CONSTRAINT FK_Employees_Departments  
FOREIGN KEY (DepartmentId)  
REFERENCES Departments(Id);
```

Think:

Plain text

CREATE → create something

ALTER → modify something

DROP → remove something

9. Transactions

Now we reach **TCL**.

A transaction is a group of operations that should be treated as **one unit of work**.

Example: transferring money:

Plain text

```
Account A: -100  
Account B: +100
```

You don't want this:

</> Plain text

A loses 100
B doesn't receive 100

Both operations should succeed, or neither should happen.

10. BEGIN TRANSACTION

Start a transaction:

</> SQL

```
BEGIN TRANSACTION;
```

Then perform operations:

</> SQL

```
UPDATE Accounts  
SET Balance = Balance - 100  
WHERE Id = 1;  
  
UPDATE Accounts  
SET Balance = Balance + 100  
WHERE Id = 2;
```

If everything is successful:

</> SQL

```
COMMIT;
```

If something goes wrong:

</> SQL

```
ROLLBACK;
```

Conceptually:

</> Plain text

```
BEGIN
  ↓
Operation 1
  ↓
Operation 2
  ↓
Success? — YES → COMMIT
          |
          NO
          ↓
        ROLLBACK
```

11. COMMIT

</> SQL

```
COMMIT;
```

Means:

Permanently accept the changes made by the transaction.

12. ROLLBACK

</> SQL

```
ROLLBACK;
```

Means:

Undo the changes made since the transaction began, or since the relevant savepoint.

Example:

</> SQL

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = Salary + 1000;

ROLLBACK;
```

The salary changes are undone.

13. SAVE TRANSACTION

You can create a **savepoint** inside a transaction.

</> SQL

```
BEGIN TRANSACTION;

UPDATE Employees
SET Salary = Salary + 1000;

SAVE TRANSACTION SalaryUpdate;

UPDATE Employees
SET DepartmentId = 5;

ROLLBACK TRANSACTION SalaryUpdate;

COMMIT;
```

The rollback goes back to the savepoint rather than cancelling the entire transaction.

Concept:

</> Plain text

```
BEGIN
|
|— Salary update
|
|— SAVEPOINT
|
|— Department update
|
|— ROLLBACK TO SAVEPOINT
    ↓
    Department update undone
    Salary update remains
```

14. DCL — Permissions

You should at least recognize these:

GRANT

Give permission:

</> SQL

```
GRANT SELECT ON Employees TO User1;
```

DENY

Explicitly deny permission:

</> SQL

```
DENY DELETE ON Employees TO User1;
```

REVOKE

Remove a previously granted/denied permission:

</> SQL

```
REVOKE SELECT ON Employees FROM User1;
```

Don't spend too much time here for a junior .NET interview unless the job specifically involves database administration.

15. DELETE vs TRUNCATE — Interview Trap

A common simplistic answer is:

"TRUNCATE can't be rolled back."

That's **not a good SQL Server interview answer**.

TRUNCATE TABLE can participate in a transaction and can be rolled back when used appropriately inside an explicit transaction.

For example:

</> SQL

```
BEGIN TRANSACTION;
```

```
TRUNCATE TABLE Employees;
```

```
ROLLBACK;
```

The important distinction is **not simply "DELETE can rollback, TRUNCATE can't."**

Focus on:

	DELETE	TRUNCATE
WHERE	✓	✗
Remove selected rows	✓	✗
Remove all rows	✓	✓
Keep table	✓	✓
DML/DDDL in SQL Server	DML	DDL
Usually faster for removing all rows	✗	✓
Identity reset	✗	✓ typically

16. DELETE and Foreign Keys

Suppose:

</> Plain text

```
Departments
  ↓
Employees
```

An employee references a department.

You can't necessarily:

</> SQL

```
DELETE FROM Departments
WHERE Id = 1;
```

if employees still reference that department.

Depending on the FK configuration, SQL Server can prevent the deletion or cascade it.

This is where:

```
</> SQL
```

```
ON DELETE CASCADE
```

can become relevant.

★ Interview Questions

Q1. What is CRUD?

Create, Read, Update, Delete — represented by INSERT, SELECT, UPDATE, and DELETE.

Q2. DELETE vs TRUNCATE?

DELETE can remove selected rows using WHERE and is DML. TRUNCATE removes all rows, doesn't support WHERE, and is classified as DDL in SQL Server.

Q3. DELETE vs DROP?

DELETE removes rows; DROP removes the database object itself.

Q4. What happens if UPDATE has no WHERE?

It updates every row that the statement targets.

Q5. What is a transaction?

A group of operations treated as a single unit of work.

Q6. COMMIT vs ROLLBACK?

COMMIT makes the transaction's changes permanent; ROLLBACK undoes them.

Q7. What is a savepoint?

A point inside a transaction that allows you to roll back part of the transaction rather than the entire transaction.

Q8. What are DDL and DML?

DDL defines/modifies database structures; DML works with the data stored in those structures.

🌐 Microsoft Learn

One-Minute Memorization

</> Plain text

```
DDL
CREATE
ALTER
DROP
TRUNCATE

DML
SELECT
INSERT
UPDATE
DELETE

TCL
BEGIN TRANSACTION
COMMIT
ROLLBACK
SAVE TRANSACTION

DCL
GRANT
```

Next: Group 9 — Keys, Constraints & Normalization, where we'll focus more deeply on PK vs UNIQUE, composite keys, FK behavior, 1NF/2NF/3NF, and normalization interview traps.

go

Group 9 — Keys, Constraints & Normalization

This is another high-priority interview group.

1. Primary Key — PK

A **Primary Key** uniquely identifies each row.

</> SQL

```
CREATE TABLE Employees
(
    Id INT PRIMARY KEY,
    Name VARCHAR(100)
);
```

Rules:

- Must be unique
- Cannot be NULL
- One PRIMARY KEY constraint per table
- Can consist of multiple columns
- SQL Server creates a unique index to enforce it. [Microsoft Learn](#)

Example:

</> Plain text

```
Id
---
1
2
3
```

You cannot have:

</> Plain text

```
1
1
```

and you cannot have:

</> Plain text

```
NULL
```

2. Foreign Key — FK

A Foreign Key establishes a relationship between tables and enforces **referential integrity**.

[Microsoft Learn](#)

Example:

</> Plain text

Departments

Id

1

2

3

Employees

Id

Name

DepartmentId ← FK

Employees.DepartmentId references:

</> Plain text

Departments.Id

Therefore:

</> SQL

```
INSERT INTO Employees
(
    Name,
    DepartmentId
)
VALUES
(
    'Ahmed',
    99
);
```

will fail if department 99 doesn't exist.

A foreign key can reference either a **primary key** or a **UNIQUE constraint** in the referenced table.

3. PK vs FK

PK	FK
Identifies a row	References another row
Must be unique	Doesn't have to be unique
Cannot be NULL	Can be NULL unless <code>NOT NULL</code>
One PK constraint per table	Multiple FKs possible
Defines entity identity	Defines relationship

Example:

</> Plain text

Department

Id ← PK

Employee

Id ← PK

DepartmentId ← FK

Many employees can have the same `DepartmentId` .

4. Composite Primary Key

A primary key can consist of multiple columns.

Example:

</> Plain text

StudentCourse

StudentId

CourseId

You could define:

</> Plain text

PRIMARY KEY (StudentId, CourseId)

This means:

</> Plain text

```
StudentId + CourseId
```

must be unique as a **combination**.

So this is allowed:

</> Plain text

StudentId	CourseId
1	10
1	20
2	10

But this isn't:

</> Plain text

1	10
1	10

← duplicate combination

Individual columns don't have to be unique. [Microsoft Learn](#)

Typical use

Many-to-many relationships:

</> Plain text

```
Students
  ↓
StudentCourses
  ↑
Courses
```

5. UNIQUE Constraint

UNIQUE means:

Values cannot be duplicated.

Example:

</> SQL

Email VARCHAR(200) UNIQUE

So:

</> Plain text

ahmed@gmail.com
ahmed@gmail.com

isn't allowed.

But unlike a PK, a `UNIQUE` constraint can allow `NULL` in SQL Server. SQL Server's documented behavior allows one `NULL` value per unique-constrained column. [🌐 Microsoft Learn](#)

Example

</> Plain text

Employees

Id PK
Email UNIQUE
Phone UNIQUE

One table can have multiple `UNIQUE` constraints, while it has only one PK constraint.

6. PK vs UNIQUE

🔥 Very common interview question.

	PRIMARY KEY	UNIQUE
Uniqueness	✓	✓
NULL	✗	✓
Number per table	One	Multiple
Identifies entity	Usually yes	Not necessarily

PRIMARY KEY

UNIQUE

Can be referenced by FK



SQL Server creates a unique index to enforce both types of uniqueness constraints. [Microsoft Learn](#) +1

7. NOT NULL

Means:

This column must have a value.

</> SQL

```
Name VARCHAR(100) NOT NULL
```

This fails:

</> SQL

```
INSERT INTO Employees (Name)
VALUES (NULL);
```

8. DEFAULT

Provides a value automatically if one isn't supplied.

</> SQL

```
IsActive BIT DEFAULT 1
```

So:

</> SQL

```
INSERT INTO Employees (Name)
VALUES ('Ahmed');
```

can result in:

</> Plain text

Name	IsActive
Ahmed	1

9. CHECK Constraint

Defines a rule for valid values.

Example:

</> SQL

```
Salary DECIMAL(10,2)
CHECK (Salary >= 0)
```

This should fail:

</> Plain text

```
Salary = -500
```

Another example:

</> SQL

```
Age INT CHECK (Age >= 18)
```

CHECK vs FK

CHECK:

</> Plain text

```
"Is this value logically valid?"
```

FK:

</> Plain text

```
"Does this referenced value exist in another table?"
```

Microsoft explicitly distinguishes them this way.  Microsoft Learn

10. Referential Integrity

Suppose:

</> Plain text

```
Department 1
  ↓
Employee 5
```

If Employee 5 references Department 1, you shouldn't be able to delete Department 1 and leave an invalid reference.

The FK protects against this.

SQL Server can use different actions:

</> Plain text

```
NO ACTION
CASCADE
SET NULL
SET DEFAULT
```

For example:

</> SQL

```
ON DELETE CASCADE
```

means deleting the parent can automatically delete the related child rows. [Microsoft Learn](#)

11. One-to-One Relationship

Example:

</> Plain text

```
User
----
Id

UserProfile
```

```
-----  
Id  
UserId
```

One user has one profile.

You normally enforce the one-to-one relationship by making the FK unique.

Conceptually:

</> Plain text

```
User.Id  
  ↑  
UserProfile.UserId  
  UNIQUE
```

Without uniqueness, multiple profiles could reference the same user.

12. One-to-Many

Most common relationship.

</> Plain text

```
Department  
  1  
  |  
  |  
  N  
Employees
```

One department:

</> Plain text

```
DepartmentId = 1
```

can appear many times:

</> Plain text

```
Employee 1 → Department 1  
Employee 2 → Department 1
```

Employee 3 → Department 1

13. Many-to-Many

You normally need a junction table.

Example:

Plain text

Students

|
|

StudentCourses

|
|

Courses

StudentCourses :

Plain text

StudentId

CourseId

Often:

Plain text

PRIMARY KEY (StudentId, CourseId)

14. Normalization

Normalization is about designing tables to:

Reduce unnecessary duplication and prevent data anomalies.

The most important levels for interviews:

Plain text

1NF
2NF
3NF

15. First Normal Form — 1NF

A table is in **1NF** when each column contains atomic values and you don't store repeating groups in one field.

Bad:

</> Plain text

Student

Name | PhoneNumbers

Ahmed | 010..., 011..., 012...

Better:

</> Plain text

Student

Id

Name

StudentPhones

StudentId

Phone

Instead of:

</> Plain text

Phone1

Phone2

Phone3

16. Second Normal Form — 2NF

2NF is mainly relevant when you have a **composite key**.

It requires:

The table is in 1NF and every non-key attribute depends on the **whole** composite key.

Example:

</> Plain text

StudentCourse

StudentId

CourseId

StudentName

CourseName

Grade

Suppose:

</> Plain text

PRIMARY KEY(StudentId, CourseId)

Problems:

</> Plain text

StudentName → depends only on StudentId

CourseName → depends only on CourseId

Grade → depends on StudentId + CourseId

So StudentName and CourseName don't depend on the whole key.

Split them:

</> Plain text

Students

StudentId

StudentName

Courses

CourseId

CourseName

```
StudentCourses
-----
StudentId
CourseId
Grade
```

Now the data is properly separated.

17. Third Normal Form — 3NF

3NF deals with **transitive dependencies**.

Simple interview definition:

Every non-key column should depend on the key, the whole key, and nothing but the key.

Example:

</> Plain text

```
Employee
-----
EmployeeId
EmployeeName
DepartmentId
DepartmentName
```

Assume:

</> Plain text

```
EmployeeId → DepartmentId
DepartmentId → DepartmentName
```

Therefore:

</> Plain text

```
EmployeeId → DepartmentName
```

indirectly.

DepartmentName belongs to the Department table.

Better:

</> Plain text

Employees

EmployeeId

EmployeeName

DepartmentId

Departments

DepartmentId

DepartmentName

★ 18. The Famous Normalization Formula

Remember this:

1NF

Atomic values

2NF

No partial dependency

3NF

No transitive dependency

Or:

</> Plain text

1NF → atomic

2NF → whole key

3NF → nothing but the key

19. What Problems Does Normalization Prevent?

Three important anomalies:

Insert anomaly

You can't insert some information without unrelated information.

Update anomaly

The same information exists in multiple places, so you must update it everywhere.

Delete anomaly

Deleting one record accidentally removes information you still needed.

Example:

</> Plain text

EmployeeId | Employee | Department | DepartmentPhone

If 50 employees belong to the same department, the department phone is duplicated 50 times.

Change the phone?

You need to update many rows.

Normalization separates:

</> Plain text

Employees

Departments

20. Should We Always Normalize?

No.

In real systems, sometimes **denormalization** is intentionally used for:

- Performance
- Reporting
- Analytics
- Reducing expensive joins

So don't answer:

"Normalization is always better."

Better:

"Normalization reduces redundancy and improves data integrity, but controlled denormalization can sometimes improve read performance."

Interview Questions

Q1. Can a table have multiple primary keys?

No.

It can have only one PK constraint, but that PK can contain multiple columns.

Q2. Can a table have multiple UNIQUE constraints?

Yes.

For example:

 Plain text

```
Id      → PK
Email   → UNIQUE
Username → UNIQUE
Phone   → UNIQUE
```

Q3. Can FK be NULL?

Yes, unless the FK column is defined as `NOT NULL`.

Q4. Does FK automatically create an index?

No.

This is an important SQL Server detail.

A PK/UNIQUE constraint generates an index, but a foreign key does **not automatically** create one. An index on the FK can nevertheless be useful for joins and referential checks.  Microsoft Learn

Q5. Can FK reference something other than a PK?

Yes.

It can reference a `UNIQUE` constraint as well.  Microsoft Learn

Q6. What's the difference between 2NF and 3NF?

2NF removes partial dependencies on part of a composite key.

3NF removes dependencies where a non-key column depends on another non-key column.

Q7. What is a composite key?

A key consisting of multiple columns whose combination uniquely identifies a row.

Final Cheat Sheet

Plain text

PRIMARY KEY

- unique + NOT NULL
- one per table

UNIQUE

- unique
- multiple allowed
- NULL allowed in SQL Server

FOREIGN KEY

- establishes relationship
- enforces referential integrity
- doesn't automatically create an index

CHECK

- validates a logical condition

DEFAULT

- supplies a default value

Plain text

1NF → Atomic values

2NF → No partial dependency

3NF → No transitive dependency

Next: Group 10 — Views, Stored Procedures & Functions. This one is particularly important for **SQL Server/.NET** backend interviews.